

High-Level Takeaways

The OS has one fundamental job: make shared hardware usable by multiple, mutually distrustful programs.

Everything in the course is a variation on this theme. Processes, scheduling, virtual memory, file systems — they're all answers to the same question asked about different resources: how do you share something safely and efficiently?

Takeaway 1: Abstraction is the core technique

The OS repeatedly takes something ugly and hardware-specific and wraps it in a cleaner interface.

- A file descriptor hides whether you're writing to a disk, a pipe, or the console.
- A virtual address hides where in physical RAM your data lives.
- A process hides the fact that the CPU is being shared.

The price of abstraction is a layer of indirection; the benefit is that programs become portable, composable, and easier to reason about. As Lampson put it: "All problems in computer science can be solved by another level of indirection."

Takeaway 2: Isolation requires hardware cooperation

The OS can't enforce protection by itself.

- Virtual memory requires an MMU.
- Kernel/user mode separation requires a privileged bit in the CPU (CPL in x86).

The pattern is always the same: **the OS sets up a data structure or register, and the hardware enforces it.**

- Page tables are set up by the OS; the MMU walks them.
- The IDT is set up by the OS; the CPU jumps into it on a fault.

This OS-configures / hardware-executes division is one of the most important structural ideas in the course.

Takeaway 3: Every design involves a tradeoff, usually the same tradeoff

You'll see the same tensions everywhere:

- **Performance vs. simplicity:** coarse-grained locks are easy to reason about but limit parallelism; fine-grained locks are fast but introduce deadlock risk.
- **Responsiveness vs. throughput:** RR gives great response time but poor turnaround; FCFS is the opposite.
- **Space vs. time:** flat page tables are fast to walk but huge; multilevel page tables are compact but require more pointer-following.
- **Transparency vs. efficiency:** mmap gives programs a clean view of files as memory, but the abstraction has hidden costs; raw read/write gives more control.

Recognizing which tradeoff is being navigated is more valuable than memorizing which option wins.

Takeaway 4: The slow path is the dangerous path

This comes up repeatedly.

- A TLB hit costs almost nothing; a TLB miss requiring a 4-level page table walk costs several memory accesses; a page fault requiring a disk read costs millions of times more.
- An uncontested mutex is cheap; a contended one causes a context switch.
- A cache hit is fast; a cache miss is slow.

The lesson: **understand your fast path and protect it.** When the slow path happens too often, you get thrashing, livelock, and collapsed throughput.

Takeaway 5: Concurrency bugs are qualitatively different from regular bugs

A regular bug is deterministic — same input, same wrong output, every time. A race condition may appear once every thousand runs, disappear when you add print statements (because they change timing), and be impossible to reproduce in a debugger. This is why the course emphasizes following Dahlin's rules rigidly rather than reasoning about correctness from scratch each time. The rules aren't bureaucracy — they're the accumulated wisdom of people who learned the hard way that clever concurrent code is almost always wrong concurrent code.

CS 5600 Final Review — Topic Outline

1. Introduction to Operating Systems (Week 1)

- What is an OS? Definition: creates an easier-to-program machine for processes
 - Two roles: **resource management** (multiplexing, sharing, isolation) and **hardware abstraction**
 - Key abstractions: files, file descriptors, virtual memory, processes/scheduling
 - Why study systems: performance, ubiquity of ideas, skill-building
-

2. Processes (Week 2)

- Definition: an instance of a running program
 - Two key abstractions: **logical control flow** and **private address space**
 - Illusion of multiprocessing; context switching mechanics
 - Multicore: kernel schedules processes across cores
 - Process states: Running, Stopped, Terminated
 - Unix Process API:
 - getpid(), getppid()
 - fork(): called once, returns twice; child gets copy of address space
 - exit()
 - wait() / waitpid(): reaping zombie children
 - exec()
 - Error handling with errno
 - Building a shell: fork() + exec() + pipes
 - Output redirection, backgrounding, piping
 - pipe(), dup2(), file descriptors
-

3. Scheduling (Week 3)

- Process state diagram (New → Ready → Running → Waiting/Terminated)
- Preemptive vs. non-preemptive scheduling
- Context switching cost (saves registers, PC, stack, page table, etc.)
- Scheduling metrics: **turnaround time**, **response time**, **throughput**, **fairness**
- Scheduling algorithms:
 - **FCFS/FIFO**: simple, no starvation, convoy effect
 - **SJF**: non-preemptive; shortest job first
 - **STCF**: preemptive SJF; best turnaround time
 - **Round-Robin (RR)**: time slices; best response time, bad turnaround for equal-length jobs
 - **Priority scheduling**: starvation risk; aging as fix
 - **MLFQ**: combines RR + priority + feedback; approximates STCF; gameable
 - **Lottery / Stride scheduling**: proportional-share; deals with starvation; unpredictable latency
- I/O and scheduling: importance of small time slices for I/O-bound processes
- Predicting future: EWMA (exponentially weighted moving average)
- How Linux schedules: CFS (Completely Fair Scheduler), based on stride scheduling

- Policy vs. mechanism split
-

4. Concurrency and Threads — Introduction (Week 4)

- Concurrency: multiple things happening "at the same time"
 - Sources: interleaved execution on single CPU, true parallelism on multicore, interrupts
 - Reasons to use concurrency: program structure, responsiveness, performance
 - **Threads** vs. processes: threads share memory, have own stack/registers (TCB)
 - POSIX thread interface: `thread_create`, `thread_exit`, `thread_join`
 - Global vs. local vs. static variables in threads
 - **Race conditions**: non-deterministic outcomes from shared state
 - Three problematic cases: concurrent reads/writes, concurrent updates (e.g., `x++`)
 - Sequential consistency: not always guaranteed in hardware (store buffers, caches, out-of-order execution)
-

5. Synchronization (Week 5)

- **Critical sections**: code requiring atomic access to shared state
 - Properties: mutual exclusion, progress, bounded waiting
 - **Locks / Mutexes**: `acquire()` / `release()`; POSIX: `pthread_mutex_*`
 - Using locks: linked list example, producer/consumer example
 - **Invariants**: what must hold before `release()`
 - **Condition variables**: `cond_wait()`, `cond_signal()`, `cond_broadcast()`
 - `cond_wait` atomically releases the mutex and sleeps
 - **Always use while, not if**, when waiting
 - **Semaphores**: historical/legacy — don't use in this class
 - **Monitors**: mutex + condition variables; language-level or programming convention
 - Dahlin's 6 Rules for thread programming:
 1. Always do things the same way
 2. Always use monitors
 3. Always hold the lock when using condition variables
 4. Acquire at start, release before return
 5. Always use while, not if
 6. Never sleep()
 - 4-step design: identify concurrency units → identify shared objects → create locks/CVs → write methods
 - Examples: readers/writers database, shared locks
-

6. Synchronization Continued — Deadlock & Other Issues (Week 6)

- **Deadlock:** 4 necessary conditions (all must hold): mutual exclusion, hold-and-wait, no preemption, circular wait
 - Deadlock handling options:
 1. Ignore it
 2. Detect and recover (resource graph, kill/rollback)
 3. Banker's algorithm (safe states)
 4. Avoid preconditions (lock ordering — partial order on all locks)
 5. Static/dynamic detection tools
 - **Livelock:** processes keep backing off, no progress
 - **Starvation:** low-priority thread never runs
 - **Priority inversion:** Mars Pathfinder example; solutions: bump priority, disable interrupts
 - Atomicity-violation and order-violation bugs
 - Performance vs. complexity: coarse vs. fine-grained locking
 - Other synchronization mechanisms: barriers, **spinlocks** (TAS and test-and-test-and-set), reader-writer locks, RCU (read-copy-update)
 - Deterministic multithreading
-

7. Virtual Memory — Introduction (Week 7)

- Benefits: programmability/transparency, protection, efficient resource use, sharing
 - VA → PA translation; hardware (MMU) configured by OS
 - Bit manipulation review: hex, binary, powers of 2
 - **Paging:** divide memory into fixed-size pages (typically 4KB = 2^{12})
 - **VPN** (virtual page number) + **offset** → PPN + offset = PA
 - **Page table:** array mapping VPN → PPN; one per process
 - Page table entry (PTE): valid/present bit, PPN, dirty bit, permission bits (U/S, R/W)
 - Size problem: flat page table is too large (e.g., 512GB for 48-bit addresses)
 - **Multilevel page tables:** sparse tree; only allocate entries in use
 - x86-64: 4-level (L1–L4), 9+9+9+9+12 bits
 - %cr3 holds physical address of L1 table
 - Practice: VPN/PPN bit calculations, page table walks
-

8. Virtual Memory — x86, TLBs, Page Faults (Week 8)

- x86-64 addresses: 48-bit virtual (canonical), 52-bit physical
- Page table structures: L1–L4, PTE bits (dirty, accessed, present, U/S, R/W, PS)
- Large pages: 2MB (skip L4), 1GB (skip L3 + L4)
- Page table walking practice
- **TLB** (Translation Lookaside Buffer): hardware cache for VA→PA
 - Managed by hardware (x86) or software (MIPS)
 - TLB miss $\neq$ page fault; page fault $\neq$ TLB miss
 - TLB flush on %cr3 load; INVLPG for individual entries

- **Where does the OS live?** Kernel mapped into top of every process's address space; protected by U/S bits
 - **Page faults:** not-present OR protection violation
 - Trap frame, IDT entry #14, %cr2 holds faulting VA, error code
 - Uses: overcommitting memory (swap to disk), mmap(), copy-on-write (fork), DSM
 - **Page replacement policies:** FIFO, LRU, CLOCK (approximates LRU)
 - **Thrashing:** working set > available memory; shed load with working sets or page fault frequency
-

Virtual Memory: Why Does It Matter?

The short answer is that virtual memory solves three distinct problems simultaneously, and it's worth being concrete about each.

Problem 1: Programs would clash without it. Without virtual memory, if you run two programs at once, what stops Program A from reading or writing Program B's memory? Nothing — it's chaos. Virtual memory gives each process its own private address space, so when your browser and your text editor both use address 0x5000, they're talking about completely different physical memory locations. The OS, via the page table, enforces that they can never see each other's data. This isolation is the foundation of everything else the OS promises you.

Problem 2: Memory would be a nightmare to manage. Without virtual memory, a program has to know at compile time exactly where in physical RAM it will live. If another program is using those addresses, you're stuck. Virtual addresses decouple the programmer's view of memory from what the hardware actually has. The compiler can always generate code that starts at address 0x0, and the OS figures out at runtime where in physical RAM the pages actually go.

Problem 3: You couldn't have more memory than you physically have. Virtual memory allows the OS to use disk as a slow extension of RAM. A process can have a 16GB virtual address space on a machine with only 8GB of physical RAM — the OS pages out less-used pages to disk and brings them back in on demand. From the program's perspective, it has all the memory it asked for.

The deep insight: **if you cannot name something, you cannot use it.** Virtual memory is the mechanism that controls what names (addresses) are valid for a given process, and therefore what it can touch. Everything else — protection, sharing, efficient memory use — flows from this idea.

When Does a Page Fault Happen? When Does the OS Read from Disk?

This is where a lot of confusion lives, so it's worth separating the concepts carefully.

Two totally different reasons for a page fault

A **page fault** is the hardware's way of saying "I tried to translate this virtual address and something went wrong." That "something" is one of two very different situations:

Case 1: Protection violation. The page is in memory, but the access is illegal — for example, a user-level process writing to a read-only page, or accessing kernel memory. The PTE is present (P=1), but the permission bits (U/S, R/W) don't allow this operation. The OS will almost always kill the process here (segfault).

Case 2: Page not present. The PTE's present bit is 0. The page simply isn't in physical RAM right now. The OS page fault handler runs, brings the page in from somewhere, updates the PTE, and restarts the instruction. This is the case that involves reading from disk.

When is the present bit 0?

There are a few reasons a valid page might not be present:

- **Demand paging / lazy allocation.** The OS doesn't actually allocate physical memory when a process asks for it with malloc. It waits until the process actually touches the address. The first touch triggers a page fault, the OS allocates a physical page, zeroes it, and maps it in. No disk involved — just lazy physical allocation.
- **Page swapped to disk.** When physical memory is full, the OS has to evict some pages to disk (a swap file or swap partition). When a process later accesses that page, the present bit is 0, the OS reads it back from disk, and resumes the process. This is the "reading from disk" scenario.
- **Memory-mapped files (mmap).** The file's contents are the backing store. Accessing a page that hasn't been loaded yet triggers a fault, and the OS reads that chunk of the file from disk into a physical page.

What actually happens during "swapping out"

An important clarification: **the OS does not save a process's entire memory when it gets deprioritized for the CPU.** That's a common misconception.

When the scheduler context-switches a process out, it only saves the **CPU registers** (the context: PC, stack pointer, general-purpose registers) into the PCB/TCB. The process's physical memory pages stay in RAM. Swapping is a separate, independent mechanism.

Swapping only happens when the OS is running low on physical memory, and a replacement policy (like CLOCK/LRU) picks cold pages to evict. The sequence is:

1. OS is running low on physical RAM
2. Page replacement algorithm picks a victim page (ideally one that hasn't been accessed recently)

3. If the victim page is **dirty** (has been written to since it was loaded), write it to the swap device on disk. If it's **clean** (e.g., a code page that can be re-read from the original executable), just discard it — it can always be reloaded.
4. Mark the PTE for that page as not-present ($P=0$), possibly storing where on disk the page now lives
5. When the process that owned that page later tries to access it, it gets a page fault, the OS reads the page back from disk, and restores the mapping

The cost is severe — make sure they feel it

From the notes: to keep disk-backed paging from degrading performance by more than 10%, only about 1 in 100,000 memory accesses can be a page fault. Miss that bar, and you hit **thrashing** — a process keeps faulting, the disk is constantly being hit, and what felt like memory now runs at disk speed (milliseconds instead of nanoseconds).

The key insight: **virtual memory is nearly free when pages are in RAM** (a TLB hit costs almost nothing). It's only when you start going to disk that the abstraction becomes expensive — and when it does, it's catastrophically expensive.

Mental Model

Think of physical RAM as a cache for disk. Virtual memory is the addressing mechanism that makes this transparent to programs. Page faults are cache misses. Like all caching, it works beautifully when there's good temporal locality, and breaks down (thrashing) when the working set doesn't fit.

9. I/O (Week 9)

- **mmap()**: maps file into virtual address space; faster than read/write (avoids kernel↔user copies)
- I/O architecture: CPU communicates with devices via:
 - Port-mapped I/O (PMIO): inb/outb instructions, separate I/O address space
 - Memory-mapped I/O (MMIO): loads/stores to special physical addresses
 - Interrupts
 - Shared memory / DMA
- **Polling vs. interrupts:**
 - Polling: wastes CPU, but low latency; interrupts: efficient but risk of receive livelock
 - Adaptive approach: start with interrupts, switch to polling if livelock
- **DMA** (Direct Memory Access): device reads/writes main memory directly; CPU sets up buffer descriptor list
- **Device drivers**: abstraction layer between kernel and hardware; bugs are dangerous
- **Synchronous vs. asynchronous I/O**: blocking vs. non-blocking interfaces
- **User-level threading**: cooperative (yield-based) or preemptive (signal-based)

- **Disks:** platters, tracks, sectors, cylinders; seek phases (speedup, coast, slowdown, settle); writes are harder than reads
-

10. File Systems (Week 12)

- What a FS does: maps filename + offset → disk block; provides persistence, naming, mapping
 - Key distinction: **files** (data) vs. **directories** (filename→filenumber mappings)
 - Components: **superblock, data bitmap, inode bitmap, inode list, data blocks**
 - **Inodes:** per-file metadata (permissions, size, timestamps, link count, block pointers)
 - 10 direct, 1 single-indirect, 1 double-indirect pointers
 - Multi-level index (similar to multilevel page tables)
 - **Reading a file** (open()/read()): traverse path directory-by-directory, following inodes
 - **Writing a file** (write()/create()): traverse path, allocate inode + data blocks, update bitmaps
 - **Deleting a file** (unlink()): traverse path → decrement link count → free data bitmap bits → free inode bitmap bit → remove directory entry (11 I/Os for a simple case)
 - **Free space management:** bitmap (simple, used in Lab 5); also free lists, B-trees
 - **LBA** (Logical Block Addressing): block number → physical location abstracted by drive
 - **Hard links vs. soft links:** hard links share an inode (link count); soft links (symlinks) are separate inodes that point to the original path
-

Important Connections Between Topics

Processes → Scheduling → Concurrency: the same CPU problem at different scales.

- A process gives each program the illusion of having the CPU to itself.
- Scheduling is how the OS manages the reality that multiple processes share one CPU.
- Concurrency is what happens when you expose that sharing more directly — threads share not just the CPU but memory, which makes the sharing much harder to reason about safely.

Virtual Memory → File Systems: the same indirection idea applied to different storage.

- Both translate a logical identifier (virtual address / filename) into a physical location (RAM location / disk block).
- Both use multi-level tree structures to handle the fact that the space is sparse — multilevel page tables and inode indirect blocks solve the same problem.
- Both use bitmaps to track what's free.
- Page faults and file system cache misses are both cases where the OS has to fetch data from a slower tier of storage.

Virtual Memory → Concurrency: why processes don't trash each other's memory. Students often wonder why we need locks when processes seem isolated. The answer is that virtual memory

provides isolation *between processes*, but threads within the same process share an address space by design. The isolation that virtual memory provides is exactly what you lose when you introduce threads.

Scheduling → I/O: you can't understand one without the other. The reason scheduling is hard in practice is I/O. A purely CPU-bound workload is easy to schedule; it's I/O-bound processes that create the interesting tradeoffs. Blocking on disk is what turns a STCF scheduler from optimal to one that causes starvation. DMA and interrupts exist precisely to avoid wasting CPU time waiting for slow devices. The whole point of asynchronous I/O is to let the scheduler do useful work while a process waits.

Context switching → Page tables → TLBs: a chain of performance concerns. Context switching is not free partly because of TLBs.

- When you switch processes, you load a new `%cr3`, which flushes the TLB.
- All the cached VA→PA translations that took effort to build up are gone.

This is one concrete reason why thread context switches are cheaper than process context switches — threads within a process share a page table, so `%cr3` doesn't change and the TLB stays warm.

The policy/mechanism split is everywhere.

- Scheduling: the mechanism is context switching (the OS can swap any time); the policy is MLFQ or CFS (deciding *when* to swap).
- Virtual memory: the mechanism is paging (the OS can move any page); the policy is LRU/CLOCK (deciding *which* page to evict).
- File systems: the mechanism is the inode/block structure; the policy is how free space is managed and how the buffer cache is evicted.

Keeping these separate is what makes systems flexible — you can change the policy without rebuilding the mechanism.

